MODALIDADE RESUMO EXPANDIDO (REX) – GRADUAÇÃO

ARQUITETURA CLOUD-NATIVE PARA SISTEMAS ESCALÁVEIS: INTEGRAÇÃO DE MICROSSERVIÇOS, SERVERLESS E OBSERVABILIDADE

CLOUD-NATIVE ARCHITECTURE FOR SCALABLE SYSTEMS: INTEGRATING MICROSERVICES, SERVERLESS COMPUTING, AND OBSERVABILITY

Gabriel Ribeiro de Camargo Miranda[1]; Deivison Shindi Takatu[2]; Glauco Todesco[3]

[1] Discente de Engenharia de Software; [2],[3] orientadores. Afiliações e contatos pendentes.

RESUMO: Este trabalho propõe uma arquitetura cloud-native para sistemas escaláveis no contexto de pesquisa de Iniciação Científica. A investigação é aplicada, exploratória e qualitativa, com revisão bibliográfica não sistemática e modelagem. Como resultados parciais, o repositório reúne descrição arquitetural, diagramas e matriz de decisões. A proposta combina aplicação web, API Gateway, serviços serverless e em contêineres, persistência relacional e NoSQL, mensageria e observabilidade. Não há protótipo nem medições; as decisões de projeto ainda requerem validação experimental.

Palavras-chave: Arquitetura cloud-native; Computação serverless; Escalabilidade; Microsserviços; Observabilidade.

ABSTRACT: This paper proposes a cloud-native architecture for scalable systems within an ongoing undergraduate research project. The applied, exploratory, and qualitative study combines a non-systematic literature review with architectural modeling. Partial results documented in the repository include an architecture description, diagrams, and a decision matrix. The proposal combines a web application, API Gateway, serverless and containerized services, relational and NoSQL persistence, messaging, and observability. No prototype or measurements are available; the design decisions require experimental validation.

Keywords: Cloud-native architecture; Microservices; Observability; Scalability; Serverless computing.

1 INTRODUÇÃO

A ampliação de serviços digitais aumenta a necessidade de sistemas que suportem variações de carga, integração entre componentes e mudanças frequentes. A adoção de nuvem, por si só, não garante elasticidade, disponibilidade ou evolução independente: essas propriedades dependem de decisões sobre fronteiras dos serviços, comunicação, dados e operação. A literatura sobre microsserviços associa a decomposição adequada à evolução independente, mas também aponta custos de integração, testes e observabilidade distribuída (NEWMAN, 2021; DI FRANCESCO; LAGO; MALAVOLTA, 2019).

O problema investigado é como estruturar uma arquitetura em nuvem que favoreça a evolução e a escalabilidade sem transferir a complexidade para a operação, a segurança e o monitoramento. A pesquisa é desenvolvida no contexto de Iniciação Científica e, até o momento, produziu documentação e modelos arquiteturais, sem implementação funcional ou avaliação experimental. Essa condição delimita o alcance das conclusões e torna pertinente apresentar os resultados como parciais.

O objetivo deste trabalho é propor uma arquitetura cloud-native que relacione requisitos não funcionais a componentes, responsabilidades e tecnologias de nuvem. Especificamente, busca-se organizar decisões sobre microsserviços, execução serverless e em contêineres, comunicação assíncrona, persistência, segurança e observabilidade, além de definir uma base para validação futura. A contribuição desta etapa é o conjunto rastreável de descrição arquitetural, diagramas de componentes e fluxos e matriz de decisões existente no repositório.

2 METODOLOGIA

A pesquisa é aplicada, exploratória e qualitativa. O procedimento combina levantamento bibliográfico não sistemático, análise documental dos materiais do projeto e modelagem de uma proposta arquitetural. Não há protocolo de revisão sistemática registrado; por isso, o levantamento é tratado como exploratório e não pretende esgotar a literatura.

Primeiro, foram reunidas fontes sobre microsserviços, computação serverless, contêineres, segurança e arquitetura em nuvem. Em seguida, os objetivos do projeto foram relacionados a requisitos não funcionais de escalabilidade, disponibilidade, manutenibilidade, segurança, observabilidade e controle de custos. Esses requisitos

orientaram a definição de componentes, responsabilidades e alternativas tecnológicas. A arquitetura e os fluxos foram registrados em diagramas Mermaid e em descrição textual, acompanhados por uma matriz de decisão.

A verificação nesta etapa é documental: examina a coerência entre problema, requisitos e decisões registradas. Não foram implementados protótipo, testes de carga ou experimentos com usuários. A validação futura deverá implementar uma prova de conceito e medir, em cenários controlados, latência, throughput, taxa de erro, comportamento sob falhas e custo operacional.

3 REFERENCIAL TEÓRICO

Microsserviços organizam uma aplicação em serviços com responsabilidades delimitadas e possibilidade de implantação independente. A definição das fronteiras deve refletir capacidades do domínio, evitando apenas distribuir um monólito sem reduzir o acoplamento (NEWMAN, 2021). Um mapeamento sistemático da área também aponta benefícios de evolução e escalabilidade seletivas, junto a desafios de integração, testes e observabilidade (DI FRANCESCO; LAGO; MALAVOLTA, 2019).

Na computação serverless, o provedor administra parcela relevante da infraestrutura e executa funções sob demanda. Revisões da área descrevem aplicações adequadas a eventos e cargas variáveis, mas registram limitações de inicialização, duração, depuração, portabilidade e dependência do provedor (SHAFIEI; KHONSARI; MOUSAVI, 2019; JOHN; GUPTA, 2021; LI et al., 2021; WEN et al., 2022). Estudos sobre impacto econômico e carregamento de contêineres reforçam que o modelo de execução deve ser escolhido conforme o perfil de cada componente (ADZIC; CHATLEY, 2017; BROOKER et al., 2023).

A comunicação assíncrona por filas e tópicos pode reduzir dependências temporais e absorver variações de demanda, desde que a solução trate duplicidade, idempotência, retentativas e mensagens que não possam ser processadas. Segurança e observabilidade também devem ser consideradas desde o desenho: autenticação, autorização, criptografia, isolamento, logs, métricas e rastreamento precisam acompanhar os fluxos distribuídos (INDRASIRI; SIRIWARDENA, 2020; AWS, s.d.-e). Esses princípios orientam a proposta, mas sua efetividade ainda depende de validação no protótipo.

4 RESULTADOS E DISCUSSÃO

Os artefatos do projeto são uma descrição dos componentes, diagramas Mermaid e uma matriz de decisões. A arquitetura proposta inclui aplicação web em S3 e CloudFront, API Gateway e Cognito, processamento em Lambda e ECS/Fargate, PostgreSQL e DynamoDB, mensageria SQS/SNS e telemetria via CloudWatch e rastreamento distribuído (AWS, s.d.-a; AWS, s.d.-b; AWS, s.d.-c; AWS, s.d.-d).

A proposta prevê Lambda para tarefas reativas e curtas e ECS/Fargate para serviços que demandem maior controle de execução. A persistência relacional ou NoSQL será escolhida conforme os dados e seus padrões de acesso. São hipóteses de projeto, não resultados de comparação empírica.

O Quadro 1 relaciona requisitos, decisões e verificações futuras. Como a distribuição de componentes pode ampliar a complexidade de integração e operação, a avaliação deverá considerar segurança e custo além de escalabilidade (DI FRANCESCO; LAGO; MALAVOLTA, 2019; AWS, s.d.-e).

Quadro 1 – Requisitos, decisões arquiteturais e validação prevista

Requisito	**Decisão proposta**	**Verificação futura**
Escalabilidade	Lambda e ECS/Fargate com expansão conforme a carga	Medir latência, throughput, erros e consumo de recursos em cenários controlados.
Segurança	Cognito, autorização por tokens, TLS, IAM e isolamento de rede	Revisar configuração, permissões e ameaças do fluxo entre componentes.
Persistência	RDS PostgreSQL e DynamoDB conforme o padrão de dados	Avaliar consistência, concorrência, integridade e adequação aos acessos.
Observabilidade	Logs estruturados, métricas, alertas e rastreamento	Injetar falhas e verificar detecção e rastreabilidade ponta a ponta.
Resiliência	SQS/SNS, retentativas, backoff e idempotência	Exercitar duplicidade, reprocessamento e mensagens não processáveis.

Fonte: Elaborado pelos autores com base nos artefatos de arquitetura do projeto.

Não há protótipo, dados experimentais ou medições de desempenho. Portanto, não é possível afirmar redução de latência, aumento de throughput, economia ou melhoria de disponibilidade. O resultado desta etapa é explicitar decisões e riscos para orientar implementação e avaliação.

5 CONSIDERAÇÕES FINAIS

A pesquisa organizou uma proposta cloud-native e documentou seus componentes, fluxos e decisões. Sua contribuição nesta etapa é um artefato conceitual rastreável para orientar a implementação e planejar a avaliação.

Sem protótipo e validação empírica, benefícios de escalabilidade, resiliência e custo permanecem hipóteses. Os próximos passos são revisar a arquitetura com os orientadores, implementar uma prova de conceito e medir seu comportamento sob carga e falhas. Os resultados deverão orientar os ajustes da proposta.

REFERÊNCIAS

ADZIC, G.; CHATLEY, R. Serverless computing: economic and architectural impact. In: *ESEC/FSE 2017*. New York: ACM, 2017. p. 884-889. DOI: 10.1145/3106237.3117767.

AWS. *Amazon API Gateway Developer Guide*. Disponível em: https://docs.aws.amazon.com/apigateway/. Acesso em: 17 set. 2026. (s.d.-a).

AWS. *Amazon CloudFront Developer Guide*. Disponível em: https://docs.aws.amazon.com/cloudfront/. Acesso em: 17 set. 2026. (s.d.-b).

AWS. *Amazon Cognito Developer Guide*. Disponível em: https://docs.aws.amazon.com/cognito/. Acesso em: 17 set. 2026. (s.d.-c).

AWS. *AWS Lambda Developer Guide*. Disponível em: https://docs.aws.amazon.com/lambda/. Acesso em: 17 set. 2026. (s.d.-d).

AWS. *AWS Well-Architected Framework*. Disponível em: https://docs.aws.amazon.com/wellarchitected/. Acesso em: 17 set. 2026. (s.d.-e).

BROOKER, M. et al. On-demand container loading in AWS Lambda. *arXiv*, 2023. Disponível em: https://arxiv.org/abs/2305.13162. Acesso em: 17 set. 2026.

DI FRANCESCO, P.; LAGO, P.; MALAVOLTA, I. Architecting with microservices: a systematic mapping study. *The Journal of Systems and Software*, v. 150, p. 77-97, 2019. DOI: 10.1016/j.jss.2019.01.001.

INDRASIRI, K.; SIRIWARDENA, P. *Microservices Security in Action*. Shelter Island: Manning Publications, 2020.

JOHN, J.; GUPTA, S. A survey on serverless computing. *arXiv*, 2021. Disponível em: https://arxiv.org/abs/2106.11773. Acesso em: 17 set. 2026.

LI, Z. et al. The serverless computing survey: a technical primer for design architecture. *arXiv*, 2021. Disponível em: https://arxiv.org/abs/2112.12921. Acesso em: 17 set. 2026.

NEWMAN, S. *Building Microservices: Designing Fine-Grained Systems*. 2. ed. Sebastopol: O'Reilly Media, 2021.

SHAFIEI, H.; KHONSARI, A.; MOUSAVI, P. Serverless computing: a survey of opportunities, challenges and applications. *arXiv*, 2019. Disponível em: https://arxiv.org/abs/1911.01296. Acesso em: 17 set. 2026.

WEN, J. et al. Rise of the planet of serverless computing: a systematic review. *arXiv*, 2022. Disponível em: https://arxiv.org/abs/2206.12275. Acesso em: 17 set. 2026.